

**MACROS ARE
EVERYWHERE**

I. Lesson

1. Declarative Macros (macro_rules!)

```
macro_rules! say_hello {  
    () => {  
        println!("Hello!");  
    };  
    ($name:expr) => {  
        println!("Hello, {}!", $name);  
    };  
}
```

-> *We will focus on that in the exercises today.*

2. Procedural Macros

A) Function-like Macros

```

● ● ●
// In a proc-macro crate
use proc_macro::TokenStream;

#[proc_macro]
pub fn make_answer(_item: TokenStream) ->
TokenStream {
    "fn answer() -> u32 { 42 }".parse().unwrap()
}

// Usage
make_answer!();

fn main() {
    println!("The answer is: {}", answer()); // 42
}

```

Look like function calls but more powerful.

B) Derive Macros

```

● ● ●
// Built-in derive macros
#[derive(Debug, Clone, PartialEq)]
struct User {
    name: String,
    age: u32,
}

fn main() {
    let user = User {
        name: "Alice".to_string(),
        age: 30,
    };

    println!("{:?}", user); // Debug works!
    let user2 = user.clone(); // Clone works!
}

```

Auto-implement traits for structs/enums.

C) Attribute Macros

```

● ● ●
// Custom attribute macro
#[proc_macro_attribute]
pub fn route(attr: TokenStream, item: TokenStream)
-> TokenStream {
    // Transform the function based on attributes
    item
}

// Usage
#[route(GET, "/users")]
fn get_users() {
    // Handler code
}

```

Add functionality to items.

3. Built-in Macros

A) Printing & Formatting

```
println!("Hello, {}!", "world");  
print!("No newline");  
eprintln!("Error message"); // stderr  
eprint!("Error without newline");  
format!("String: {}", value); // returns String
```

B) Assertions

```
assert!(condition);  
assert_eq!(left, right);  
assert_ne!(left, right);  
debug_assert!(condition); // only in debug builds
```

C) File & Module System

```
include!("file.rs"); // Include file content  
include_str!("file.txt"); // Include as &str  
include_bytes!("file.bin"); // Include as &[u8]
```

4. Specialized Macros

A) Testing Macros

```
● ● ●  
#[test]  
fn test_something() {  
    assert_eq!(2 + 2, 4);  
}  
  
#[should_panic]  
fn test_panic() {  
    panic!("Expected panic");  
}  
  
#[ignore]  
fn expensive_test() {  
    // Skipped by default  
}
```

B) Benchmarking

```
● ● ●  
#[bench]  
fn bench_something(b: &mut Bencher) {  
    b.iter(|| {  
        // Code to benchmark  
    });  
}
```

C) Configuration Macros

```
● ● ●  
#[cfg(target_os = "linux")]  
fn linux_only() {}  
  
#[cfg(debug_assertions)]  
fn debug_only() {}  
  
#[cfg_attr(feature = "serde", derive(Serialize))]  
struct Data {}
```


5. Common Third-Party Macros

A) Serde (Serialization)

```
use serde::{Serialize, Deserialize};

#[derive(Serialize, Deserialize)]
struct User {
    name: String,
    age: u32,
}
```

C) Thiserror (Error Handling)

```
#[derive(Error, Debug)]
enum MyError {
    #[error("IO error: {0}")]
    Io(#[from] std::io::Error),
}
```

B) Tokio (async)

```
#[tokio::main]
async fn main() {
    // Async code
}
```

D) Actix-web (Web Framework)

```
#[get("/users")]
async fn get_users() -> impl Responder {
    // Handler
}
```

Leptos

The view! Macro

Procedural Macro (simple)

```
● ● ●  
  
// You write:  
view! {  
    <h1>"Hello World"</h1>  
}  
  
// Expands to:  
leptos::leptos_dom::html::h1()  
    .child(  
        leptos::leptos_dom::text::Text::new("Hello World")  
    )
```

Procedural Macro (nested)

```
● ● ●  
  
// You write:  
view! {  
    <div>  
        <h1>"Title"</h1>  
        <p>"Paragraph"</p>  
    </div>  
}  
  
// Expands to:  
leptos::leptos_dom::html::div()  
    .child(  
        leptos::leptos_dom::html::h1()  
            .child(leptos::leptos_dom::text::Text::new("Title"))  
    )  
    .child(  
        leptos::leptos_dom::html::p()  
            .child(leptos::leptos_dom::text::Text::new("Paragraph"))  
    )
```


II. Exercices

Let's code!

Your turn!